

Table 1: A comparative analysis of IoT protocols

Features	CoAP	MQTT	AMQP	DDS
Messaging pattern	request/response	publish-subscribe	request/response; publish-subscribe	publish-subscribe
Architecture	tree	tree	star	bus
Transport	UDP	TCP, MQTT-S: UDP	TCP	UDP or TCP
Network layer	IPv6	IPv4 or IPv6	IPv4 or IPv6	IPv6
QoS level	2 levels	3 levels	3 levels	23 levels
Communication scope	Device-to-device	Device-to-cloud	Device-to-device, Device-to-cloud, Cloud-to-cloud	Device-to-device, Device-to-cloud, Cloud-to-cloud
Addressing	URI	topic only	queue, topic/Routing Key	topic/key
Security	DTLS, IPsec	TLS	SASL/TLS	TLS, DTLS, DDS security
Interoperability	Semantic	Foundational	Structural	Semantic
[Broker/server] implementation language	Java, C, C#, C++, Erlang, Go, Python, JavaScript, Ruby, Rust, Swift	Java, C, C#, C++, JavaScript, Erlang, Go, Lua, Python	Java, C, C#, C++, Python, Ruby	Ada, Java, C, C#, C++, Python, Scala, Lua, Pharo, Ruby
Header size	4 bytes	2 bytes	8 bytes	16 bytes
Governing body	IETF	OASIS	OASIS	OMG

Data Distribution Service (DDS)

DDS is a middleware protocol for data-centric connectivity from the object management group, which provides low-latency, data connectivity, extreme reliability, and a scalable architecture for business and mission-critical IoT applications. This protocol supports multicasting techniques in data transmission and high-quality QoS in small memory footprint devices and applications. The messaging model in DDS consists of two interface layers — data-centric, publish-subscribe (DCPS) and data local, reconstruction layer (DLRL). The DCPS layer is responsible for binding the values of data objects within an application during the publish/subscribe process. The DLRL is an optional layer used for integrating DDS at the application level.

There are two main constructs in DCPS — publisher with DataWriter and subscriber with DataReader. A publisher uses the DataWriter to bind values of data objects for each defined data type. It is responsible for data distribution while adhering to the QoS policies, if any. The DataWriter describes the data that requires publishing when an application needs to write data associated with data objects. A subscriber

receives published data, based on a predefined set of QoS policies, from an application. An application uses a DataReader attached to a subscriber to retrieve the data from the latte. It subscribes to data described by a DataReader, which is provided by a known subscriber. Publishing and subscribing to data objects is done by using topics. A topic is associated with a name, data type and one or more QoS policies related to the data. The QoS policies are often used to manage how the data is distributed in the messaging model. DDS provides 23 different QoS levels with features such as security, durability, reliability, and many

others. Figure 7 presents a high-level overview of the communication model of the DCPS layer in DDS.

Basic similarities among the several IoT messaging protocols that exist today (Table 1) suggest that they have the potential to coexist throughout the design and deployment of IoT systems. Given the diversity among IoT device types and the protocols they can support, it is not rare for IoT systems to employ several protocols for IoT data exchange. Since messaging protocols are key components for the connectivity of IoT devices, understanding their strengths and weaknesses is important while choosing an appropriate one. **END** 

References

- [1] IoT architecture (<https://www.hindawi.com/journals/jece/2017/9324035/>)
- [2] Types of IoT connections (<https://www.sam-solutions.com/blog/internet-of-things-iot-protocols-and-connectivity-options-an-overview/>)
- [3] MQTT publish/subscribe architecture (<https://mqtt.org/>)
- [4] CoAP messaging model (<https://dzone.com/articles/coap-protocol-step-by-step-guide>)
- [5] CoAP message flows (<https://devopedia.org/constrained-application-protocol>)
- [6] AMQP architecture (https://www.researchgate.net/figure/Advanced-message-queueing-protocol-architecture_fig8_305987854)
- [7] DDS data-centric model (<https://www.dds-foundation.org/what-is-dds-3/>)

By: Divya Singh

The author is a networking geek with eight years of experience in the IT industry as a domain network consultant. She has an insatiable curiosity for DevOps and cloud. She loves travelling, reading, and writing on social issues and travel experiences. She is an aspiring best-selling author.